

MST Weight & Bottleneck

Minimum Spanning Trees, Bottleneck Edge · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

For a weighted graph, report the minimum spanning tree weight, its bottleneck (the heaviest edge in the tree), and the number of connected components. You are given starter code with two functions. `bottleneck` is almost complete but has a bug, and `count_components` is an empty stub. Fix the bug and implement the stub.

What to Compute

- **MST Weight:** the total weight of the minimum spanning tree (or forest).
- **Bottleneck:** the maximum weight among the chosen tree edges.
- **Components:** the number of connected components.

Input / Output Format

Input: Line 1: N M . Next M lines: an undirected weighted edge u v w .

Output: Three lines: MST Weight, Bottleneck, Components.

Examples

Example 1

Input: 5 6 0 1 1 0 2 3 1 2 1 1 3 4 2 3 2 3 4 5

Output:

MST Weight: 9
Bottleneck: 5
Components: 1

Explanation: The MST uses 0-1 (1), 1-2 (1), 2-3 (2) and 3-4 (5) for weight 9; its heaviest edge is 5, and all five vertices form one component.

Example 2

Input: 3 3 0 1 1 1 2 2 0 2 3

Output:

MST Weight: 3
Bottleneck: 2
Components: 1

Explanation: The MST 0-1 (1) and 1-2 (2) has total 3 and a heaviest edge of 2; one component.

Constraints

- Build the minimum spanning forest with Kruskal's algorithm.
- The bottleneck is the largest weight among chosen edges.
- Components = N minus the number of chosen edges.

Your Task

- Task 1 - Fix the bug. bottleneck is flagged with a comment, but the bug's location is not given. Find the single bug so it returns the LARGEST chosen edge weight, not the smallest.
- Task 2 - Implement count_components from scratch: N minus the number of chosen edges.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys

def find(parent, x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def mst_edges(n, edges):
    edges = sorted(edges, key=lambda e: (e[2], e[0], e[1]))
    parent = list(range(n))
    chosen = []
    for u, v, w in edges:
        ru = find(parent, u)
        rv = find(parent, v)
        if ru != rv:
            parent[ru] = rv
            chosen.append((u, v, w))
    return chosen

# BUG: this function contains a single bug. The bottleneck is the LARGEST edge weight
# in the spanning tree, but this version reports the smallest.
def bottleneck(chosen):
    best = chosen[0][2] if chosen else 0
    for u, v, w in chosen:
        if w < best:
```

```

        best = w
    return best

# TODO: implement this. With (n-1) tree edges the graph is one component; in general the
# number of components is n minus the number of chosen edges. Currently a stub.
def count_components(n, chosen):
    return 0

def main():
    data = [int(x) for x in sys.stdin.read().split()]
    n = data[0]; m = data[1]
    idx = 2
    edges = []
    for _ in range(m):
        edges.append((data[idx], data[idx + 1], data[idx + 2]))
        idx += 3
    chosen = mst_edges(n, edges)
    weight = sum(w for u, v, w in chosen)
    print("MST Weight: " + str(weight))
    print("Bottleneck: " + str(bottleneck(chosen)))
    print("Components: " + str(count_components(n, chosen)))

main()

```

C++

```

// MST Lab 5 - MST Weight & Bottleneck (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <algorithm>
using namespace std;

struct Edge {
    int u;
    int v;
    long long w;
};

bool edgeLess(const Edge& a, const Edge& b) {
    if (a.w != b.w) {
        return a.w < b.w;
    }
    if (a.u != b.u) {
        return a.u < b.u;
    }
    return a.v < b.v;
}

int findRoot(vector<int>& parent, int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

```

```

vector<Edge> mstEdges(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), edgeLess);
    vector<int> parent(n);
    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }
    vector<Edge> chosen;
    for (const Edge& e : edges) {
        int ru = findRoot(parent, e.u);
        int rv = findRoot(parent, e.v);
        if (ru != rv) {
            parent[ru] = rv;
            chosen.push_back(e);
        }
    }
    return chosen;
}

// BUG: this function contains a single bug. The bottleneck is the LARGEST edge weight
// in the spanning tree, but this version reports the smallest.
long long bottleneck(const vector<Edge>& chosen) {
    long long best = chosen.empty() ? 0 : chosen[0].w;
    for (const Edge& e : chosen) {
        if (e.w < best) {
            best = e.w;
        }
    }
    return best;
}

// TODO: implement this. The number of components is n minus the number of chosen edges.
// Currently a stub.
int countComponents(int n, const vector<Edge>& chosen) {
    return 0;
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<Edge> edges(m);
    for (int i = 0; i < m; i++) {
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    }
    vector<Edge> chosen = mstEdges(n, edges);
    long long total = 0;
    for (const Edge& e : chosen) {
        total += e.w;
    }
    cout << "MST Weight: " << total << "\n";
    cout << "Bottleneck: " << bottleneck(chosen) << "\n";
    cout << "Components: " << countComponents(n, chosen) << "\n";
    return 0;
}

```

Java

```
// MST Lab 5 - MST Weight & Bottleneck (STARTER)
import java.util.*;

public class Main {
    static class Edge {
        int u, v;
        long w;
        Edge(int u, int v, long w) {
            this.u = u;
            this.v = v;
            this.w = w;
        }
    }

    static Comparator<Edge> edgeOrder = (a, b) -> {
        if (a.w != b.w) {
            return Long.compare(a.w, b.w);
        }
        if (a.u != b.u) {
            return Integer.compare(a.u, b.u);
        }
        return Integer.compare(a.v, b.v);
    };

    static int find(int[] parent, int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    static List<Edge> mstEdges(int n, List<Edge> edges) {
        edges.sort(edgeOrder);
        int[] parent = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
        List<Edge> chosen = new ArrayList<>();
        for (Edge e : edges) {
            int ru = find(parent, e.u);
            int rv = find(parent, e.v);
            if (ru != rv) {
                parent[ru] = rv;
                chosen.add(e);
            }
        }
        return chosen;
    }

    // BUG: this function contains a single bug. The bottleneck is the LARGEST edge weight
    // in the spanning tree, but this version reports the smallest.
    static long bottleneck(List<Edge> chosen) {
        long best = chosen.isEmpty() ? 0 : chosen.get(0).w;
        for (Edge e : chosen) {
            if (e.w < best) {

```

```

        best = e.w;
    }
}
return best;
}

// TODO: implement this. The number of components is n minus the number of chosen edges.
// Currently a stub.
static int countComponents(int n, List<Edge> chosen) {
    return 0;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    List<Edge> edges = new ArrayList<>();
    for (int i = 0; i < m; i++) {
        edges.add(new Edge(sc.nextInt(), sc.nextInt(), sc.nextLong()));
    }
    List<Edge> chosen = mstEdges(n, edges);
    long total = 0;
    for (Edge e : chosen) {
        total += e.w;
    }
    System.out.println("MST Weight: " + total);
    System.out.println("Bottleneck: " + bottleneck(chosen));
    System.out.println("Components: " + countComponents(n, chosen));
}
}

```

Sample Run

Sample Input

```

5 6
0 1 1
0 2 3
1 2 1
1 3 4
2 3 2
3 4 5

```

Sample Output

```

MST Weight: 9
Bottleneck: 5
Components: 1

```